FULL RECOPILACION DE PARCIALES
(2024-2025) BY ROY FIORILO

Teoría de Algoritmos (TB024)

17 de junio de 2025

Parcial 1º
Oportunidad

Apellido y Nombres: .....

Padrón: .....

Pregunta	1	2	3	4	5	Total	
Calificación							Corrigió:

(ENTREGAR CADA EJERCICIO EN UNA HOJA DIFERENTE)

1- **DIVISIÓN Y CONQUISTA**. Realizar un algoritmo que resuelva la multiplicación de dos matrices cuadradas de orden n mediante división y conquista. Recordemos la definición: $C = A \times B$ si $C_{ij} = \Sigma_k A_{ik}. B_{kj} \forall i,j = 1..n$. Para simplificar el algoritmo se puede suponer que n es una potencia de 2. Indicar el orden de complejidad del algoritmo.

2- **PROGRAMACIÓN DINÁMICA.** El INVAP debe transportar una carga extremadamente peligrosa desde Ezeiza hasta Bariloche. El traslado se hará por tierra y en etapas, ya que no se pueden hacer más de 400km por día y no se puede viajar de noche. Se conocen las diferentes alternativas de rutas y lugares donde parar, también se conoce el costo del seguro para cada camino posible. (El pésimo estado de las rutas es un riesgo más, y las aseguradoras lo han tenido en cuenta). Por ejemplo, para la primera etapa hay tres caminos posibles: Ezeiza – Gral Villegas, Ezeiza-Bolívar ó Ezeiza-Azul. Los costos de seguro son de $57, $82 y $39 respectivamente. Realizar un algoritmo mediante programación dinámica que resuelva el problema calculando el menor costo de seguro. Indicar el orden de complejidad del algoritmo.

3- **GREEDY.** Realizar un algoritmo Greedy que resuelva el problema planteado en el punto 2. Indicar si la solución obtenida será óptima o no.

4- **BACKTRACKING.** Realizar un algoritmo que resuelva el problema de las 8 reinas mediante backtracking. Recordemos que el problema consiste en ubicar 8 reinas en un tablero de ajedrez de manera que no hay dos reinas en la misma fila, columna o diagonal. Indicar las estructuras de datos utilizadas para representar el problema y la solución.

5- **PROGRAMACIÓN LINEAL.** Realizar un modelo de programación lineal que resuelva el problema planteado en el punto 2 ó en el punto 4. Indicar la cantidad de variables y restricciones del modelo.

Teoría de Algoritmos (TB024) 27 de mayo de 2025

Parcial 1º
Oportunidad

Apellido y Nombres: ..... Padrón: .....

Pregunta	1	2	3	4	5	Total	
Calificación							Corrigió:
Supervisión							Supervisó:

(ENTREGAR CADA EJERCICIO EN UNA HOJA DIFERENTE)

1- **REDES DE FLUJO.** El sistema energético argentino está dividido en 9 regiones según se muestra en la tabla de la derecha. Para cada región se indica la Generación de energía en esa región prevista para mañana, y la demanda esperada de esa región para mañana. Ambos datos se pueden considerar constantes conocidas. Desde cualquier región i se puede enviar energía a la región j, con un costo de Cij pesos por MWh transportado. Además, cada línea tiene una capacidad máxima de CAPij MWh por día.

	Generación (MWh)	Demanda (MWh)
NEA	Ga	Da
MOA	Gb	Db
GBA	Gc	Dc
Centro	Gd	Dd
Patagonia	Ge	De
Litoral	Gf	Df
Comahue	Gg	Dg
PBA	Gh	Dh
Cuyo	Gi	Di

Realizar un algoritmo que, mediante Ford Fulkerson, determine cuánta energía transportar entre cada par de zonas para cubrir las demandas. Se deben detallar todos los pasos (incluyendo los del algoritmo de FF).

2- **GREEDY.** Plantear un algoritmo Greedy que resuelva el problema anterior.

3- **PROGRAMACIÓN LINEAL.** Plantear un modelo de Programación Lineal que encuentre la solución más económica al problema anterior.

4- **PROGRAMACIÓN DINÁMICA**. Tenemos el problema del delivery. Es similar al problema de la mochila, pero además del peso Wi de cada elemento, debemos considerar su volumen Vi y su costo Ci. La mochila tiene una capacidad máxima de C kilos y T litros. Además, por una restricción del seguro, el costo total de los productos transportados no puede superar los M pesos. Desarrollar un algoritmo de PD que resuelva el problema.

5- **REDUCCIONES**. Marco, el cadete de la empresa, está en problemas. Le pidieron que compre un paquete de sw para resolver el problema del Ciclo Hamiltoniano de manera eficiente. Como siempre está distraído, compró el que resuelve el problema del viajante. El vendedor le dijo que si resuelve TSP, también puede resolver fácilmente el ciclo hamiltoniano. Determinar si el vendedor tiene razón o hay que echar a Marco. Recordemos que:
Ciclo Hamiltoniano: Dado un grafo incompleto, determinar si existe un camino tal que recorra todos los vértices del grafo, sin pasar dos veces por el mismo vértice, y vuelva al origen.
TSP (Problema de decisión): Dado un grafo completo y un número k, determinar si existe un camino que recorra todos los vértices del grafo, sin pasar dos veces por el mismo vértice y vuelva al origen, con un costo menor o igual a k.

Teoría de Algoritmos (TB024)

5 de noviembre de 2024

Parcial
1º Oportunidad

Apellido y Nombres: .....

Padrón: .....

Pregunta	1	2	3	4	5	Total	
Calificación							Corrigió:
Supervisión							Supervisó:

1) BACKTRACKING (Sugerido): El teorema de los cuadrados de Lagrange especifica que cualquier número entero no negativo n puede expresarse como la suma de cuatro cuadrados de enteros no negativos: $n = w^2 + x^2 + y^2 + z^2$

Ejemplos: $42 = 4^2 + 4^2 + 3^2 + 1^2$ $310 = 17^2 + 4^2 + 2^2 + 1^2$

Desarrollar un algoritmo que, dado el valor de n, indique cuales son los cuadrados que lo componen. ¿Cuál es la complejidad de dicho algoritmo?

2) GREEDY (Sugerido): Alan Turing y su equipo están trabajando a contrarreloj para descifrar el código de comunicación de los submarinos nazis. Se sabe que usan un algoritmo de sustitución simple: cada letra la reemplazan por otra letra específica. Los espacios y los signos de puntuación no se cambian. La clave (es decir, qué letra reemplaza a cada una) se cambia diariamente por otra clave similar. A las 6 AM reciben el primer mensaje del día con la nueva clave y quieren descifrarlo cuanto antes, para que sea de utilidad. Para ello se centran en las palabras de cuatro letras del mensaje. Toman una lista de 20 palabras del mensaje. También tienen un "diccionario" donde están casi todas las palabras de 4 letras que suelen usar los submarinos.

Desarrollar un algoritmo que encuentre una clave que traduzca las palabras de la lista y por lo menos 15 de ellas se encuentren en el diccionario.

3) PROGRAMACIÓN DINÁMICA: Paolo Casanova se va a tomar el próximo fin de semana para descansar en la playa. Como quiere cargar poco equipaje, sólo llevará una mochila. La mochila tiene una capacidad de L litros y puede resistir un peso máximo de C kilos. Tiene una lista de todos los elementos que desearía llevar, pero no le caben todos en la mochila. Para cada elemento sabe su peso, su volumen y el "beneficio" que le daría llevarlo. Los ítems 1 y 2 de la lista son dos guías de observación de aves. No tiene sentido llevar ambas, pero debe llevar una de las dos. Su objetivo es maximizar el beneficio de los objetos que lleve en la mochila.

Desarrollar un algoritmo de PD que encuentre la solución óptima.

4) PROGRAMACIÓN LINEAL: Paolo Casanova está dedicado a su nuevo hobby, la observación de aves. Dispone de 2 días y desea planificar salidas a diferentes lugares cercanos para ver la mayor cantidad de especies posibles. Hay 10 especies en particular que desea encontrar y 6 lugares en donde podría encontrarlas. Algunos lugares se recorren en medio día, otros requieren un día completo, como se indica en la siguiente tabla:

Lugar	Duración (días)	Especies									
		Gaviota cangrejera	Colorada	Gaviotin Sudam.	Verderón	Doradito	Tachurí	Ratona	Cachilo canela	Playerito	Lechuza
Mar Chiquita	½	X	X		X			X		X	
Parque Lago	½	X				X	X	X		X	
Arroyo el Cangrejo	½	X	X	X			X				
Playa	½			X	X	X	X	X		X	
Est. Nahuel Ruca	1		X	X	X		X		X	X	
Laguna Los Horcones	1	X	X			X	X		X		X

Plantear un modelo de PL que determine qué lugares visitar para encontrar la mayor cantidad posible de especies en el tiempo disponible.

5) REDUCCIONES: Demostrar que el problema del cambio de monedas no es más difícil que el de suma de subconjuntos (Subset Sum). ¿Es posible demostrarlo en sentido opuesto? ¿Por qué? (El problema de Subset sum dice: Dado un grafo G = (V,E) cuyos vértices tienen un peso; encontrar un subconjunto de vértices cuyos pesos sumen exactamente k)

Teoría de Algoritmos (TB024) 26 de noviembre de 2024

Parcial
2º Oportunidad Apellido y Nombres: ..... Padrón: .....

Pregunta	1	2	3	4	5	Total	
Calificación							Corrigió:
Supervisión							Supervisó:

1. Implementar un algoritmo que dado un Grafo no dirigido nos devuelva un conjunto de vértices que representen un mínimo Vertex Cover del mismo.

2. Dado un número K, se quiere obtener la mínima cantidad de operaciones para llegar de 0 a K, siendo que las operaciones posibles son: (i) aumentar el valor del operando en 1; (ii) duplicar el valor del operando. Implementar un algoritmo que, por **programación dinámica** obtenga la menor cantidad de operaciones a realizar (y cómo son dichas operaciones). Desarrollar la ecuación de recurrencia. Indicar y justificar la complejidad del algoritmo implementado.

3. Trabajamos para el mafioso Arnook, que es quien tiene la máxima influencia y poder en la ciudad costera de Ciudad República. Allí reina el caos y la delincuencia, a tal punto que quien termina organizando las pequeñas mafias locales no es otro sino Arnook. En particular, nos vamos a centrar en unos pedidos que recibe de parte de dichos grupos por el control de diferentes kilómetros de la ruta costera. Cada pequeña mafia le pide a Arnook control sobre un rango de kilómetros (por ejemplo, la mafia nro 1 le pide del kilómetro 1 al 5, la mafia 2 le pide del 3 al 8, etc. . .). Si hay una mafia tomando control de algún determinado kilómetro, no puede haber otra haciendo lo mismo (es decir, no pueden solaparse). Cada mafia pide por un rango específico. Arnook no cobra por kilómetraje sino por "otorgar el permiso", indistintamente de los kilómetros pedidos. Ahora bien, esto es una mafia, no una ONG, y no debe rendir cuentas con nadie, así lo único que le interesa es maximizar la cantidad de permisos otorgados (asegurándose de no otorgarle algún lugar a dos mafias diferentes). Implementar un algoritmo **Greedy** que reciba los rangos de kilómetros pedidos por cada mafia, y determine a cuáles se les otorgará control, de forma que no haya dos mafias ocupando mismo territorio, y a su vez maximizando la cantidad de pedidos otorgados. Indicar y justificar la complejidad del algoritmo implementado. Justificar por qué el algoritmo planteado es Greedy. ¿El algoritmo da la solución óptima siempre?

4. En un hospital, se tiene un conjunto de médicos y un conjunto de pacientes. Cada médico tiene un horario con franjas horarias disponibles para citas médicas. Nuestro objetivo es emparejar médicos con pacientes de manera que se maximice el número total de citas médicas programadas. Implementar un algoritmo que resuelva dicho problema de manera eficiente. Indicar y justificar la complejidad del algoritmo implementado.

5. Un set dominante (Dominating Set) de un grafo G es un subconjunto D de vértices de G, tal que todo vértice de G pertenece a D o es adyacente a un vértice en D. El problema de decisión del set dominante implica, dado un grafo G y un número k, determinar si existe un set dominante de a lo sumo tamaño k. Demostrar que el Dominating Set Problem es un problema NP-Completo. Ayuda: recomendamos recordar Vertex Cover, que puede ser útil para esto.

Teoría de Algoritmos (TB024) 10 de diciembre de 2024

Parcial
2º Oportunidad Apellido y Nombres: ..... Padrón: .....

Pregunta	1	2	3	4	5	Total	
Calificación							Corrigió:
Supervisión							Supervisó:

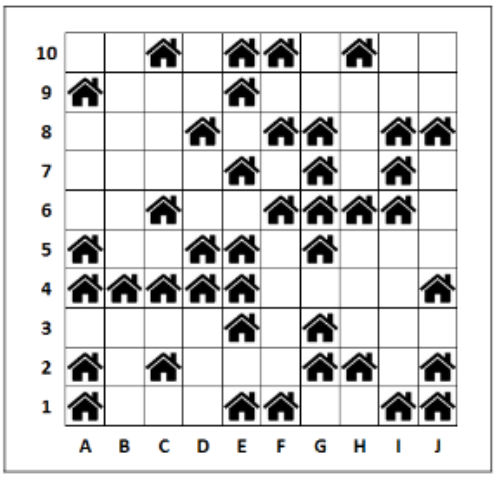
1) **REDES DE FLUJO:** Indicar si la siguiente afirmación es verdadera o falsa. Si es verdadera, indicar por qué. Si es falsa, dar un contraejemplo: "Sea G una red de flujo arbitraria, con una fuente s y un sumidero t y una capacidad entera positiva Ce en cada arista e. Sea A-B un corte mínimo s-t con respecto a esas capacidades. Si sumamos 1 a la capacidad de cada arista, el corte A-B sigue siendo mínimo con respecto a las nuevas capacidades."

2) **PROGRAMACIÓN DINÁMICA**: Se tiene un texto, formado por una sucesión de palabras y espacios en blanco. Cada palabra Wi tiene Ci caracteres. Se quiere dividir el texto en líneas, tal que la longitud de las líneas sea lo más pareja posible. Por ejemplo, si asignamos las palabras Wi,… Wk a una línea, la longitud de la línea será:

$$\left[\sum_{i=j}^{k-1}(c_i+1)\right]+c_k \le L.$$

Una línea es válida si cumple esta desigualdad. (O sea, L es la longitud máxima de la línea). A la diferencia entre el miembro izquierdo y el miembro derecho de la desigualdad se lo llama sobrante. (El sobrante es la cantidad de espacios en blanco en el margen derecho). Realizar un algoritmo que realice una partición de un texto dado en líneas tal que minimice la suma de los cuadrados de los sobrantes de todas las líneas.

3) **PROGRAMACIÓN LINEAL:** Una empresa de telefonía celular decidió colocar antenas en una nueva área a la cual quiere prestar servicios. Esa área es la que vemos en la figura de la derecha (en donde hay dibujadas casas, hay un vecindario). Si se coloca una antena en una celda, dará servicio a 9 celdas (la celda en la cual se coloca y las 8 que la rodean). Por ejemplo, si se instala una antena en B2 brindará servicios a A1, B1, C1, A2, B2, C2, A3, B3 y C3. Colocar una antena sale $CTE (el costo es el mismo en cualquier celda). La empresa sabe que es muy difícil brindar servicio a todos los vecindarios, así que quiere brindar servicio a por lo menos el 70% de los vecindarios. ¿Qué es lo mejor que se puede hacer con la información disponible?



4) **GREEDY:** Otra empresa de telefonía celular decidió colocar antenas en la misma área a la cual quiere prestar servicios. La capacidad de cobertura de las antenas es similar a la anterior. La única diferencia es que el costo de las antenas varía según la celda en que se coloquen. El costo de colocar una antena en la celda (i,j) es $CTOij. El personal de esta empresa no sabe de programación lineal, por lo que se desea realizar un algoritmo greedy que indique en qué celdas colocar antenas.

5) **REDUCCIONES:** Un set dominante (Dominating Set) de un grafo G es un subconjunto D de vértices de G, tal que todo vértice de G pertenece a D o es adyacente a un vértice en D. El problema de decisión del set dominante implica, dado un grafo G y un número k, determinar si existe un set dominante de a lo sumo tamaño k. Demostrar que el Dominating Set Problem es un problema NP-Completo. Ayuda: recomendamos recordar Vertex Cover, que puede ser útil para esto. (Vertex Cover es un subconjunto de nodos de un grafo tal que toda arista del grafo es adyacente al menos a un vértice del subconjunto)

```
Def Greedy (oportunidades_trabajo: list[int]):
    oportunidades_trabajo = Sort (Key = False)   # No Recuerdo si así se pasaba la lista descendentemente
    cont_dias_trabajados = 0
    Beneficio_obtenido = 0
    For trabajo in oportunidades_trabajo:
        if cont_dias_trabajados >= 3:
            Beneficio_obtenido += trabajo
            Cont_dias_trabajados += 1
        Else:
            cont_dias_trabajados = 0

    Return Beneficio_obtenido
```

- La complejidad del Algoritmo Greedy es $O(n \log(n))$ ya que al ordenar la lista, tenemos como cota máxima (o peor de los casos) la complejidad antes mencionada

MARTIN ROSSI

B

3

. Para comenzar Planteo mi Variable Bibolente y Nombro Datos:
$X_i = 1 \Rightarrow$ Si marcos trabaja el Dia i (Variable Bibolente)
B_i : Representa el Beneficio obtenido el Dia i (Dato del Problema)

. Mi unica Restriccion es la Inicial que marcos Trabaja solo 2 Dias Seguidos para ello: $X_i + X_{i+1} + X_{i+2} \leq 2$
Con esta Restriccion Nos aseguramos que solo Trabaja 2 Dias

. Mi funcion objetivo es maximizar el Beneficio, Para por la cual $\Rightarrow$ $Z(max) = \sum_{i}^{20} B_i * X_i$

MARTIN ROSSI

(4)

```
Def Dyc( Vector)
    if len(Vector)==-1:
        Return Vector,0:

    mitad = len(Vector)//2
    iz,Posible_iz= Dyc(Vector[:mitad])
    Der,Posible_Der = Dyc(Vector[mitad:]

    if Posible_iz==0 and Posible_Der==0:
        Andinn = dez(iz[0]-Der[0])
        if Andinn >1:
            Return iz+Der, iz[0]+1
        Return iz+Der,0
    if Posible_iz >0:
        Return iz+Der, Posible-iz
    if Posible_Der>0:
        Return iz+Der, PosibleDer

Def Febronie_natural(Vector):
    - NumFebronie = Dyc(Vector)
    Return NumFebronie
```

. Como se que Tengo el Array ordenado, En cada llamado Recursivo, corto el Array a la mitad y en cada caso donde ambas mitades hayan llegado a su caso Base, se hace una

Primer

Verificación De Si aun NO Se Encontro el Numero Desordenado.
Se hizo la Resta Se calcula la Resta absoluta De iz[0] - Der[0]
Si esto Da mayor a 1, quiere Decir que hubo iz[0] y Der[0]
falta un Numero, Por lo cual Si el Encuentra Se Devuelve
el Array Unido (iz+Der) y el Numero que Estaba Desordenado.
y luego Se va Progando la Solución hasta Ser Resuelta. Si
no Se Encontro un Numero Desordenado el Algoritmo Devuelve 0.

$T(n) = A \cdot T(B/n) + T(n^c)$

A = 2
B = 2
c = 1

~~donde la calculadora pero tuve problema tiene complejidad~~
~~O(log(n)), que es mejor que O(n) (lineal)~~

$O(n^{\log_2(A)}) = O(n^{c=1}) \Rightarrow O(n)$

$O(n^{\log_2 2^{=1}}) = O(n) \Rightarrow O(\log(n))$

MARTIN
ROSSI

(B)

(5) Primero Para Demostrar que el Problema mencionado De 2 particiones Es NP-completo, Realizo un Algoritmo que Verifique una Solución candidata Del Mi Problema En tiempo polinomial.

```
Def Verificar_2_particiones (L1: List[int], L2: List[int]):
    Suma_Beneficio_l1 = Sum(L1)
    Suma_Beneficio_l2 = Sum(L2)
    Return Suma_Beneficio_l1 == Suma_Beneficio_l2
```

Este Verificador Es de tiempo Polinomial ⇒ O(n).

Luego aprovechando la Sugerencia, Del problema De Subset Sum el cual ya Sabemos que Es NP-completo, Debemos Demostrar que Subset sum ≤p 2 particiones

1. Reformulo el Problema De Subset Sum En 2 Particiones

El problema de Subset Sum Recibe como Entrada un array con X n elementos y un T el cual Es la Suma a la cual Se Quiere Llegar formando posibles conjuntos que Sumados den T

En Nuestro caso Necesitamos que Nos Devuelva Necesariamente Dos Conjuntos que Entre Si Sumen T. Para Satisfacer la Solución De 2 particiones.

Para esto Suponemos que Tenemos Nuestro Array S

arreglo del Problema de la Paridad

La única forma de obtener Solución En Este caso es Construir un S', tal que Este compuesto por:
$S' = S \cup (2T - T)$ Donde $T = \pi$ y $T = Sum(S)$
además Una Suma Dada De Sin Si o Si por fora que haya Solución.

Vamos a Demostrarlo con un caso Posible
Si Nuestro $S = [4, 2, 3, 2, 3]$ y Nuestro T En Este caso $T = 14$, como T es Par, Vamos que Sucede cuando ocurrar Nuestro $S' = S \cup (2 \cdot 14 - \pi 4) = S \cup 14$
con Esto $S' = [4, 2, 3, 2, 3, 14]$ y si ahora Hacemos el Algoritmo De Subset Sum con $T = 14$ Tendremos Dos Soluciones (como En 2 Particiones) $\Rightarrow [4, 2, 3, 2, 3], [14]$ ambos conjuntos Suman 14 y Podemos Ya Demostrar que Subset Sum es igual o No más Difícil que 2 particiones y como Subset Sum es NP-completo, Subset Sum también lo es

~~Caso Donde T No es Par. $S = [4, 2, 3, 2, 2]$, Pero $T = 13$~~
~~Si Hacemos $S' = S \cup (2\pi - T) = S \cup (26 - 13) = S \cup 13$~~
~~$S' = [4, 2, 3, 2, 2, 13]$, Si Ponemos Este Para El Algoritmo~~
~~De Subset Sum~~

Nombre: Rebeca Pinta

Padrón: 108672

② (Greedy) (B)

Hago

del algo que pasa por costo

```
def minimizar_costo_flujo(regiones, capacidades, costos):
    grafo = Grafo(dirigido=True)
    for region, generacion, demanda in regiones:
        grafo.agregar_vertice(region)

    for region1, _, _ in regiones:
        for region2, _, _ in regiones:
            if region1 == region2:
                continue
            grafo.agregar_arista(region1, region2)
            grafo.agregar_arista(region2, region1)

    costos = [(i, j, costos(i,j)) for i in range(cantidad)]
    costos_ordenados = sorted(costos, key = lambda x: x[2])
    #ordeno las aristas por costo.

    #Voy cubriendo demandas y producción en aristas de
    #menor costo.
    flujo = {}

    for (i, j, costo) in costos_ordenados:
        if flujo[(i,j)] not in flujo:
            flujo[(i,j)] = costo * regiones[i][2]
            (i-2, j-2) = buscar_arista_en_costo((i, costos
            flujo[(i2, j-2)] = costo(i-2, j-2) * regiones[i][1]
    return flujo
```

demanda

producción

```
def -bucar- costas[ i, costos ):
    min =
    For ((j-1, j-2, costo) in costas:
        is costo < min and j-2 == i:
            min = costo
            indices=(j-1, j-2)

    return indices
```

↓

Busca el coste mínimo y región que incide en i de coste mínimo, para poder ampliar lo que debe abarcar la región i.

Regla greedy: Busco la región de coste mínimo asigno que se cumple lo demanda y luego busco lo mínimo de lo otro pero que se cumpla lo generamos

Nombre: Rebeca Dávila
Región: 208672

3. (Programación lineal)

Datos o Parámetros:

CAP_{ij}: capacidad de cada línea, de la línea que va de la región i hacia j.

C_{ij}: costo de ir a la región i a la j.

$i, j \in \{1, 2, \dots, 9\}$ (9 regiones)

G_i: generación de la región i.

D_i: Demanda de la región i.

Función objetivo: $\rightarrow$ $Z_{min} = \sum_{i=1}^{9} \cdot \sum_{j=1}^{9} X_{ij} \cdot C_{ij}$
(minimizar el costo total)

Defino el flujo de energía de la región i hacia j)
X_{ij}: flujo de energía de i hacia j

Restricciones: 1. Que no pase más flujo que la capacidad
2. Que se cumplan las demandas
3. Que se cumplan las generaciones.

Para todo (i, j):

1. $X_{ij} \leq \underbrace{C_{ij}}_{CAP_{ij}} \rightarrow$ para cada línea de conexión entre cada región i y j

2. que se cumplan las demandas:
para cada demanda:

$$D_i = \sum_{j=1}^{9} X_{ji}$$

NO se tiene en cuenta lo generado por la región

3. que se cumplan las generaciones

para cada generación de energía de cada región:

$$e_i = \sum_{j=1}^{9} X_{ij}$$

NO se tiene en cuenta lo demandado por la región

Nombre: Rebeca Anahi Druin
Padrón: 108672 B-

4. (Programación dinámica) → (minimizar el costo)

```
def Problema_delivery( elementos , C , T , M ): Python ≠ Pseudocódigo
    OPT = [ [ [ 0 for t in range( T + 1 ) ] for j in range( C + 1 ) ] for _ in
                                                    len(elementos) + 1 ]

    for i in range( 1 , len(elementos) + 1 ):
        w, v, costo = elementos [ i - 1 ]  # ( w_i , v_i , c_i )
        for c in range( 1 , C + 1 ):
            for t in range( 1 , T + 1 ):
                if c >= w and t >= v :
                    OPT[i][c][t] = min( OPT[i-1][c][t] , OPT[i-1][c-w][t-v]
                                                        + costo
                else:
                    OPT[i][c][t] = OPT[i-1][c][t]

    return OPT
```

Para este problema se desea minimizar el costo total de llevar elementos, pero además debemos respetar restricciones de volumen, peso y costo (presupuesto M).

Para el paso i (teniendo en cuenta el elemento i)

Defino:

OPT[i][c][t]: el costo mínimo teniendo en cuenta hasta el elemento i, con capacidad de peso c y t de capacidad de volumen.

$$OPT[i][c][t] = \begin{cases} \text{no poner } i \rightarrow OPT[i-1][c][t] \\ \text{poner } i \rightarrow OPT[i-1][c-w_i][t-v_i] + costo \end{cases}$$

elijo el mínimo entre poner el elemento y no

```
def delivery(elementos, C, T, M)
    OPT = delivery_problema(elementos, C, T, M) # resuelvo con programación dinámica
    elementos_a_llevar = construir_solucion(OPT, C, T, M, len(elementos), [])
    elementos_a_llevar.reverse()
    return elementos_a_llevar

def construir_solucion(OPT, c, t, M, i, solucion, elementos)
    if i==0 or c==0 or t==0:
        return solucion
    w, v, costo = elementos[i-1]
    if OPT[i-1][c][t] < M and OPT[i-1][c][t] ≤ OPT[i-1][c-w][t-v]+costo:
        return construir_solucion(OPT, c, t, M, i-1, solucion, elementos)
    elif OPT[i-1][c-w][t-v]+costo ≤ M and OPT[i-1][c-w][t-v]+costo < OPT[i-1][c][t]:
        solucion.append(elementos[i-1])
        return construir_solucion(OPT, c-w, t-v, i-1, solucion, elementos)
    else:
        return solucion
```

Complejidad

Nombre: Rebeca Anaila D'Avila
Padrón: 108672

1. (Redes de flujo)

- Se tienen 9 regiones del sistema energético argentino
- Se conoce la generación de energía (producción) y la demanda (consumo) por región i a la j.

Cada línea tiene una capacidad máxima CAP_{ij} MWh por día.

Esto se puede modelar como un problema de producción y demanda con el algoritmo de Ford-Fulkerson.

```
def determinar_flujo(regiones, capacidades, costos):
    # regiones es una lista de: (NEA, Cs, Ds)
    # costos y capacidad son diccionarios
    grafo = Grafo(dirigido = True)
    for (region, generacion, demanda) in regiones:
        grafo.agregar_vertice(region)
    # una cada region con cada region
    For (region1, -, -) in regiones:
        For (region2, -, -) in regiones:
            if region1 == region2:
                continue
                                                    Peso
                                                     ↓
            grafo.agregar_arista(region1, region2, capacidad(region1, region2))
            grafo.agregar_arista(region2, region1, capacidad(region2, region1))
    # agrego una fuente e y un sumidero z
    grafo.agregar_vertice(e)
    grafo.agregar_vertice(z)
    # Por cada demanda voy a agregar una arista desde la fuente
    # a ese vertice y por cada generación voy a agregar
    # una arista desde ese vertice al sumidero.
```

~~De la fuente sale lo que genera cada región, y al sumidero llega lo que consume.~~

```
for (region, generacion, demanda) in regiones:
    grafo.agregar_arista(S, region, demanda)
    grafo.agregar_arista(region, C, generacion)

# ahora resuelvo por ford fulkerson:
flujo, grafo_residual = ford_fulkerson(grafo, S, C)
return flujo
```

```
def ford_fulkerson(grafo, S, C)
    flujo = {}
    for v in grafo:
        for w in grafo.adyacentes(v):
            flujo[(v,w)] = 0

    grafo_residual = copiar(grafo)
    while (camino = obtener_camino(grafo_residual, S, C)) and camino
    is not None):
        peso_minimo = obtener_arista_minima(camino)
        for i in range(1, len(camino)):
            if grafo.hay_arista(camino[i-1], camino[i]):
                flujo[(camino[i-1], camino[i])] += peso_minimo
            else:
                flujo[(camino[i], camino[i-1])] -= peso_minimo
        actualizar_grafo_residual(grafo, grafo_residual, camino[i], camin
                                  peso_minimo)
    return flujo
```

existe un camino de S a T entonces tiene que existir siempre esta arista

falta implementar

~~falta devolver la solucion (como~~

1. (Slugo)

Nombre: Rebeca D'Avila

Padrón: 108672

```
def actualizar_grano_residual(grano, grano_residual, origen, destino, peso_min)
    if grano_residual.hay_arista(destino, origen):
        peso_anterior = grano_residual.peso(destino, origen)
        grano_residual.cambiar_peso(destino, origen, peso_anterior + peso_m
    else:
        grano_residual.agregar_arista(destino, origen, peso = peso_min)

    if grano_residual.hay_arista(origen, destino):
        peso_anterior = grano_residual.peso(origen, destino)
        [illegible]
        if peso_anterior >= peso_min:
            grano_residual.cambiar_peso(origen, destino, peso = peso_anter
            - peso_min)
            ↳ resto
        if peso_anterior == peso_min:
            grano_residual.borrar_arista(origen, destino)
```

3-

101694 Bogliar Nicolás

HOJA

FECHA

1) La ecuación de recurrencia va a ser

$$\max \{ pd[i-1], pd[i-2] + beneficio(i) \}$$

↳ recibo una lista con el max beneficio de cada día

```
def changer (dias):
    if dias[1] == 1 día:
        return max(dias)
    pd = [dias[0], dias[0] + dias[1]]
    for d in range (2, len(dias)):
        pd[d] = max ( pd[d-1], pd[d-2] + dias[d] )
    return reconstruccion ( pd )

def reconstruccion ( pd, dias ):
    Trabajo_index = [ ]
    for i in range ( len(pd) : -1 )
        if pd[i] != pd[i-1]
            trabajo_index.append(i)
    return Trabajo_index
```

NOTA

B | 101694 Boglia Nicolás | HOJA # | FECHA

2) def changes_greedy(dias): (*) → [0,1]
dias_trabajadas = [1, dias[0]]
for d in range(2, len(dias)):
if dias_trabajados[d-2] + 1 == dias_trabajados[d-1]:
dias_trabajados.append(d)
return dias_trabajados

(*) Por rapidez recibo una lista con el mayor beneficio de cada día.

La idea es trabajar dos días seguidos agarrando el trabajo con mayor beneficio y no trabajar el otro.

No es óptimo

↳ Contraejemplo: [1, 1, 10] ← este es el que conviene
el algoritmo agarra estos dos

NOTA

3/4 408042

GAIGIS

2

4) LO PRIMERO VOY A RECORRER SIEMPRE EMPEZANDO POR LAS LISTAS IZQ YA QUE COMO ES DE MENOR A MAYOR Y TENGO QUE BUSCAR EL NUMERO MAS CHICO, ES LA MEJOR OPCION

DE ESTA MANERA EL PRIMER NUMERO QUE ENCUENTRE QUE NO ESTE EN LA LISTA VOY A SABER QUE ES EL MAS CHICO

```
DEF  NOESTA( LISTA, P)
     SI  LISTA[P-1] != (LISTA(P) -1)
         DEVUELVE 1
     SI  LISTA[P,1] != (LISTA(P) +1)
         DEVUELVE 2
DEF  ENCONTRARMASCHICO(LISTA, i, j)
     SI  i > j                    (SI NO ENCUENTRA LA SOLUCION)
         DEVUELVE -1
     MITAD = (i + j)//2
     SI  NOESTA(LISTA, MITAD) = 1
         DEVUELVE (MITAD -1)
     SI  NOESTA(LISTA, MITAD) = 2
         DEVUELVE (LISTA[MITAD] + 1)
```

✓ ENCONTRARMASCHICO(LISTA, I, MITAD) (LLAMA DEVUELTA LA FUNCION POR SU LADO IZQ)

✓ ENCONTRARMASCHICO(LISTA, MITAD+1, J) (LLAMA DE VUELTA A LA FUNCION POR SU LADO DER)

TEOREMA MAESTRO

A = 2

B = 2

C = 0

$\log_B A = C \;=\; \log_2 2 > 0$

C (NO HACE MAS QUE COMPARAR)

$1 > 0$

POR LO TANTO LA COMPLEJIDAD EN EL PEOR DE LOS CASOS ES O(N) (OSEA TECNICAMENTE CASI SIEMPRE ES MENOR QUE O(N) A ~~LO~~ NO SER QUE SEA EL ANTEULTIMO EL NUMERO QUE FALTA)

EN RESUMEN, LO QUE HACE ESTE PROGRAMA ES VERIFICA SI EL ANTERIOR Y EL SIGUIENTE DEL MEDIO SON EL ANTERIOR Y EL SIGUIENTE DE LISTA(MEDIO), ASI VA VERIFICANDO ACHICANDO CADA VEZ MAS EL PROBLEMA HASTA ENCONTRAR LA SOLUCION. SIEMPRE ARRANCANDO POR LA IZQUIERDA

LA RELACION CON CONCURRENCIA, ES LLAMAR LA FUNCION DEVUELTA, PERO ~~[illegible]~~ PASANDOLE LA MITAD DE LA LISTA ASI SUCESIVAMENTE, RECORRIENDO TANTO MITADES IZQ Y DER DE LISTAS QUE SE DIVIDEN EN MITADES. ASI HASTA ENCONTRAR LA SOLUCION ONO (SI SE RECORREN TODAS LAS MITADES Y NO SE ENCUENTRA LA SOLUCION)

$\sum_i D_p B_i = 1$

3/4 1080 dr

GANGS

$\log_B A < C$

(b)

3) TENEMOS QUE CADA DIA TIENE DIFERENTES TRABAJOS (Y OBVIAMENTE VAA A GARDAR EL QUE MAS LE PAGAI, Y QUE NO TRABAJA 20 DIAS SEGUIDOS.

ESTO POR LOS PROXIMOS 20 DIAS

DIA 1 DIA 2 DIA 3 DIA 20

[[20,30,10], [50,20,5], [10,5,20] ..... [...]]

NOSOTROR QUEREMOS MAXIMIZAR EL DINERO QUE HACE MAZCO

~~B: $50,13 = SI ES$~~

B_i GE = BENEFICIO DEL TRABAJO i (~~DE~~ E

$D_p\{0,1\}$ = SI ES 1, TRABAJA EL DIA P

$D_p\{0,1\}$ = SI ES 1, HACE EL TRABAJO i EN EL DIA P

MAXIMIZAR $\sum_p^{20} \sum_{i}^{iGP} D_p B_i$ iGP | PARA TODO i QUE SE ENCUENTR EN EL DIA P

RESTRICCIONES

~~$\sum_i D_p B_i = 1$~~ ~~$\forall p$~~ (ESCOGER, QUE SOLO PUEDE AGARRAR UN TRABAJO POR DIA)

$\sum_i D_{pi} = 1 \quad \forall p$

~~$D_p + D_{p+1} \leq 1$~~ (NO PUEDE TRABAJAR 20 DIAS SEGUIDOS)

$D_p + D_{p+1} + D_{p+2} \leq 2$ (COMO MAXIMO TRABAJA 2 DIAS SEGUIDOS)

4/4 10 5 12

GANGIS

(B)

5) UNA DE LAS PAREJAS MAS TRICAS PASAN UN DIVORCIO

VIENES AUTO, CASAS, ETC.

NO SE PONEN DE ACUERDO EN LA MANERA DE DIVIDIRLO

EL JUEZ DICTAMINO QUE UN TASADOR PONGA

VALOR A CADA BIEN Y LUEGO SE HAGA UNA

PARTICION POR VALORES IGUALES

(PROBLEMA ABSTRACTO 2 PARTICIONES)

EL JUEZ NO SPIDE QUE ELAVORE NOS UN ALGO.

QUE DE FORMA EFICIENTE HAGA ESTE TRABAJO

DEMOSTRAR QUE LA SOLUCION PEDIDA ES

NP-COMPLETO

SUGERENCIA REALIZARLO STABLE WIZ QUE

SUBSET SUM ES NP COMPLETO

-o-o

5) BUENO, PARA SABER QUE UN ~~PROBLEMA~~ ALGORITM.

ES NP-COMPLETO ~~[redacted]~~ SE TIENE QUE PODER

VERIFICAR LA SOLUCION TIENE UN VELOCIDAD INFERIOR

O IGUAL A POLINOMIAL O(n log(n))

SERIA RECORRER UNA LISTA, GUARDAR LOS PARAMETROS

DE ESTA Y COMPARARLOS (MUY EN RESUMEN)

ENTONCES SABEMOS QUE VERIFICA DEL PROBLEMA

NOS DA UNO COMPLEJIDA O(n)

(LA CUAL ES LINEAL Y ES MENOR A LA POL. NOM. AL)

2) TENEMOS QUE PODER REDUCIRLO POR OTRO PROBLEMA NP-COMPLETO (SABEMOS QUE SUM SET ES NP-COMPL

~~SUBSETSUM ∈ P ALGORITMO~~

ALGORITMO ∈ PSUBSETCOMPLETO

3) HAGO LA REDUCCION

SABEMOS QUE SUBSET L=[2, 3, 5] T=8

SUBSET: {3,5}

NUESTRO ALGORITO HACE ALGO COMO ESTO

[12 10 4 2] ⟶ [12,2], [10,4]

ENTONCES MI IDEA E

RECIBO LA LISTA DE LAS COSAS TAZADAS, SUMO TODOS LOS NALORES QUE HAY EN LA LISTA

AGARRO ESTE NUMERO Y LO DIVIDO POR 2

Y HAGO EL SUBSETSUM 2 VECES CON ESTE NUMERO (EL QUE EST
O CON LA LISTA RESTANTE)

DIVIDO O

EJ: [12,10,4,2] -> SUMA TOTAL = 48 -> //2 = 24

SUBSTET SUM L=[12,10,4,2] I=24 -> [12,2] PARA EL DIVORCIA

LO QUE QUEDA DE LA OTRA LISTA

[10,4] PARA LA DIVORCIADA

EL PROBLEMA DE ESTA REDUCCION ES QUE SI ES IMPAR NO SE RESUELVE DE MANERA SATISFACTORIA (SE LO QUEDAN LOS ABOGADOS)

1/4 108092

GANGIS

(B)

DIA 1	DIA 2	DIA 3	DIA 4
B0 B1 B2	B0 B1 B2	B0 B1 B2	B0 B1 B2

2) [30, 20, 50], [10, 15, 30], [100, 7, 15], [40, 50, 20])

(ESTRUCTURA DE LA LISTA DE DIAS CON SUS RESPECTIVOS TRABAJOS)

~~MAYOR~~

SEGUIDO = 1 (VARIABLE QUE ME PERMITE SABER CUANDO LLEGO AL TERCERO Y SALTEARLO)

GANANCIAS TOTALES = 0 (ALMACENA LA MAXIMA GANANCIA DE CADA DIA)

FOR DIA IN LISTA_DIAS

MAYOR PAGO = 0 (GUARDA LA MAXIMA GANANCIA DEL DIA, Y DESPUES SE REINICIA)

SI SEGUIDO == 3 (YA QUE NO PUEDE LABURAR 3 DIAS SEGUIDOS)

~~PASA AL SIGUIENTE DIA~~

SEGUIDO = 1 (SE REINICIA EL CONTADOR)

PASA AL SIGUIENTE DIA (NO TRABAJA ESE DIA)

FOR B IN DIA: (LOS TRABAJOS DEL DIA (GANANCIA)

SI B > MAYOR PAGO

MAYOR PAGO = B

GANANCIAS TOTALES MAS MAYOR PAGO

+

SEGUIDO MAS 1 (SE LE SUMA UN DIA MAS A SEGUIDO)

DEVOLVE GANANCIAS TOTALES

NO, NO ES UN ALGORITMO OPTIMO → YA QUE NO VA A ENCONTRAR EL MAXIMO LOCAL

~~POR EJEMPLO~~: EN LA LISTA QUE PUSE ARRIBA DE TODO, EN EL TERCER DIA HAY UN TRABAJO DE 70. EN CAMBIO EL MAXIMO QUE PUEDO

NO, LA SOLUCIÓN OBTENIDA NO VA SER OPTIMA.
YA QUE SEGURAMENTE NO ENCUENTRE EL MAXIMO LOCAL
LA MAYORIA DE LAS VECES.

EN EL EJEMPLO QUE VIA ATRAS, LA MAYOR GANANCIA
DEL DIA 1 ES 50 Y DEL DIA 2 ES 30, POR LO TANTO EL
TOTAL ES 80, PERO EN EL DIA 3, LA MAXIMA GANANCIA
ES 100. (MAS QUE LOS 2 DIAS ANTERIORES JUNTOS)
PERO COMO ES EL TERCER DIA NO VA A TRABAJAR
~~TRABAJAR~~
TRABAJAR ESE DIA

Teoria de Algoritmos (TB024)

Parcial 1º Oportunidad

17 de junio de 2025

Apellido y Nombres: Tomás Fernandez

Padrón: 111290

Pregunta	1	2	3	4	5	Total
Calificación			B	B-	B-	(6)

Corrigió:

(ENTREGAR CADA EJERCICIO EN UNA HOJA DIFERENTE)

1- **DIVISIÓN Y CONQUISTA**. Realizar un algoritmo que resuelva la multiplicación de dos matrices cuadradas de orden n mediante división y conquista. Recordemos la definición: $C = A \times B$ si $C_{ij} = \Sigma_k A_{ik} . B_{kj}$ $\forall i,j = 1..n$. Para simplificar el algoritmo se puede suponer que n es una potencia de 2. Indicar el orden de complejidad del algoritmo.

2- **PROGRAMACIÓN DINÁMICA.** El INVAP debe transportar una carga extremadamente peligrosa desde Ezeiza hasta Bariloche. El traslado se hará por tierra y en etapas, ya que no se pueden hacer más de 400km por dia y no se puede viajar de noche. Se conocen las diferentes alternativas de rutas y lugares donde parar, también se conoce el costo del seguro para cada camino posible. (El pésimo estado de las rutas es un riesgo más, y las aseguradoras lo han tenido en cuenta). Por ejemplo, para la primera etapa hay tres caminos posibles: Ezeiza – Gral Villegas, Ezeiza-Bolívar ó Ezeiza-Azul. Los costos de seguro son de $57, $82 y $39 respectivamente. Realizar un algoritmo mediante programación dinámica que resuelva el problema calculando el menor costo de seguro. Indicar el orden de complejidad del algoritmo.

3- **GREEDY.** Realizar un algoritmo Greedy que resuelva el problema planteado en el punto 2. Indicar si la solución obtenida será óptima o no.

4- **BACKTRACKING.** Realizar un algoritmo que resuelva el problema de las 8 reinas mediante backtracking. Recordemos que el problema consiste en ubicar 8 reinas en un tablero de ajedrez de manera que no hay dos reinas en la misma fila, columna o diagonal. Indicar las estructuras de datos utilizadas para representar el problema y la solución.

5- **PROGRAMACIÓN LINEAL.** Realizar un modelo de programación lineal que resuelva el problema planteado en el punto 2 ó en el punto 4. Indicar la cantidad de variables y restricciones del modelo.

B

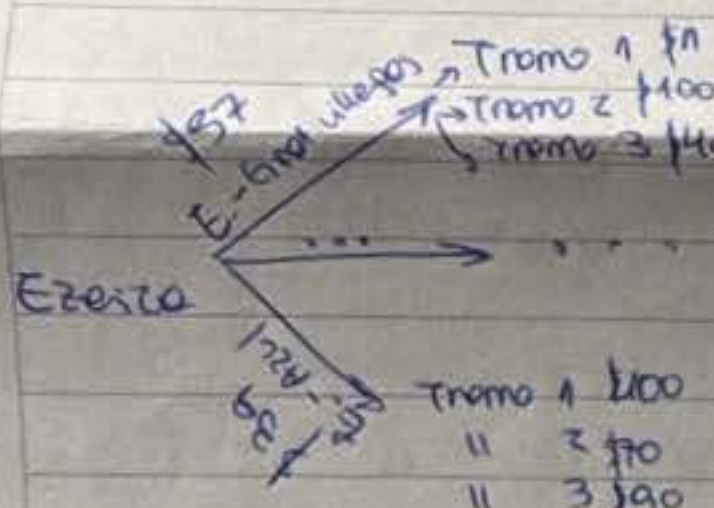
Tomás Fernandez 111290

Hoja 3

Fecha

(3) Para la resolución de este ejercicio vamos a tomar ~~usar~~ una regla Greedy ~~algoritmos~~ la cual no siempre nos va a dar la solución mas optima, ya que la característica principal de los algoritmos Greedy es que en cada paso aplique la regla.

Mi regla Greedy sera que en cada tramo que voy recorriendo, siempre elijo el de costo más barato, pero esto no siempre devolvera la solucion optima, ya que por ejemplo en el primer paso puedo elegir la opcion de 39 pesos, pero tal vez en el proximo tramo ~~la~~ el costo del más barato ~~caro~~ ~~escaso~~ del camino previo de costo $57 es 1 peso, por lo que le suma, sera menor que si elijo el camino de $39, lo explico con un dibujo para que quede mas claro.



Mi algoritmo elegira el camino Ezeiza-Azul y Tramo 2 desde alli, con un costo de $109. Pero el camino mas barato seria hacer Ezeiza-Gral. Villegas y desde alli el tramo 1, donde el costo tiene menor con un total de $58.

Por esta razón mi algoritmo no siempre devolvera la solución optima.

Notas:

Planteare el algoritmo pensando que voy a recibir un grafo donde sus vertices adyacentes serán los caminos disponibles desde ahí. Por ejemplo:



En cada paso me quedare con el vertice de menor valor.

cantidad de caminos → n, Ezeiza → vertice_inicial

```
def menor_costo(grafo, n, vertice_inicial):
    costo=0
    aux = INF
    v=vertice_inicial
    for i in range(n):
        for w in grafo.obtener_adyacente(v):
            valor = obtener_valor(w)
            if valor < aux:
                aux = valor
                v= w
        costo += aux
        aux = INF

    return costo
```

Lo recorro!

Inicializo aux después de cada iteración para que aux en infinito para que luego tome el valor mínimo.
También obtener_valor es una función que le paso un vertice y me dice su valor.

complejidad?

Notas:

Tomás Fernandez 111290 B-

Hoja: 2

Fecha:

(4) Para llevar acabo la resolucion de este problema, lo modelare como un independent set, donde cada caballero sera un vertice y las aristas entre ellos representaran si puedo llegar desde un caballero a otro. Al independent set, le pasare un parametro k igual a 8, que nos indicara si puede obtener el independent set de ese tamaño, si lo logre, esto nos indicara que puedo colocar los 8 reinas.

```
def independent_set(grafo, k):
    vertices = grafo.obtener_vertices()
    solucion = []
    return is_rec(grafo, vertices, solucion, sol_parcial, 0, k)


def es_is_valido(grafo, solucion, k):
    if len(solucion) < k:
        return False

For i in range(len(solución)):
    For v in grafo.obtener_adyacentes(solución[i]):
        if v is in solucion:
            return False

return True
```

Notas:



```
def is_rec(grafo, vertices, solucion, sol_parcial, indice, k):
    if len(sol_parcial) == k:
        if es_is_valido(grafo, sol_parcial, k):
            solucion.extend(sol_parcial)
            return True
        else:
            return False

    sol_parcial.append(vertices[indice])
    if is_rec(grafo, vertices, solucion, sol_parcial, indice+1, k):
        return True
    else:
        sol_parcial.pop()
        return is_rec(grafo, vertices, solucion, sol_parcial, indice+1, k)
```



Tomás Fernandez - 111290

Hoja: 1

Fecha:

B-

(5) Resuelvo con lineal el punto 4.

El problema de las reinas como mencione anteriormente, lo transformare en un problema de tipo Independent Set.

Para ello ~~[illegible]~~ lo modelaremos armando un grafo donde cada ~~[illegible]~~ casillero del tablero sera un vertice y los aristas que uniran los vertices (seran los cuales) on el tablero son los casilleros o los cuales puede llegar desde un casillero ij

Entonces mi Función objetivo es maximizar los vertices que pertenezcan al IS, en este ejemplo que son 8.

Función objetivo: $Z(max) = \sum_{i=1}^{n} x_i$

Variables: $x_i = 1$ Si el vertice i pertenece al IS, esto para todo i desde 1 a n. Si no pertenece vale 0.

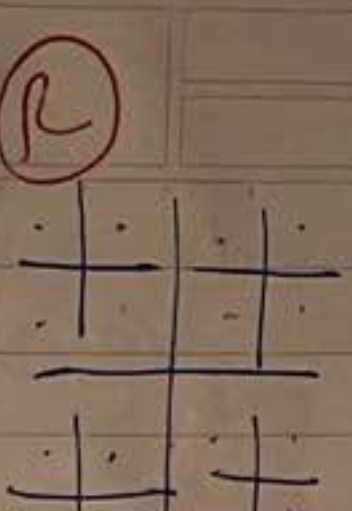
$x_{ij} = 1$ Si existe una ariste entre el vertice ij y el S. (conjunto de aristas) 0 en caso contrario. Esto para todo par ij.

Restricciones: $x_i + x_j \leq 1$ Para todo par $(i,j) \in E$, con esta restricción digo que de ese par de vertices solo uno o ninguno puede ser parte del IS, Porque tienen una ariste en común. Si existe una ariste o no, eso lo se en base a la bivalente.

Notas:

Ignacio Vicu 196930

1) def dividir_matrices(m1, m2):
IF len(m1) <= 1 or len(m2) <= 1:
return m1, m2
Fin = columns(m1)
mid = Fin // 2



asumo que columns() me devuelve
el largo (la cantidad de columnas)

~~arriba_izq = dividir_matriz[0, mid][0, mid~~
arriba_izq = dividir_matriz(m1[0,mid][0,mid], m2[0,mid][0,mid])
arriba_der = dividir_matriz(m1[mid,0],[0,mid], m2[mid,0],[0,mid])
hacer lo mismo (con los parametros que corresponda) para
abajo_izq y abajo_der

return multiplicar(arriba_izq, arriba_der, abajo_izq, abajo_der)

def multiplicar(...):
res = arriba_izq[0] * arriba_izq[1]
res = arriba_der[0] * arriba_der[1] + res
res = abajo_izq[0] * abajo_der[1] + res
res = abajo_der[0] * abajo_der[1] + res

return res

Bueno, no está tan claro, en parte porque yo tampoco lo tengo tan claro, pero una matriz siempre se puede subdividir en 4 partes y eso nos va a devolver otra matriz de dimensión $n/2$ hasta llegar al caso base de tener una unidad que deberemos multiplicar con la unidad que haya en la misma posición en la otra matriz.

~~Ahora que lo escribo, me parece que lo que estoy haciendo~~

O al menos, esa es la idea general del código, dividir el problema hasta un caso más sencillo y después realizar las multiplicaciones necesarias.

El teorema maestro nos dice que:

$$T(n) = aT\left(\frac{n}{b}\right) + F_n$$

a = cant subproblemas recursivos = 4

b = en cuanto se reduce = 2

F_n = Trabajo fuera de recursión = 1

$\log_b a > c$, es decir $\log_2 4 > 1$

por lo tanto la complejidad = $O(n^{\log_2(4)})$

Ignacio Vian 106930 (R)

2) Estando que estamos trabajando con un grafo pesado y ~~dirigido~~ y queremos resolver este problema de camino mas barato por programación dinamica.

```
def camino_mas_barato(grafo):
    n = Cantidad_vertices(grafo)
    dP[0] * n
    Prev = [-1] * n

    For i in range(n)
        For j in adj[i]
            iF dP[j] + Peso(j) < dP[i]:
                dP[i] = dP[j] + Peso(j)
                prev[i] = j

    Fin : dP.index(dP.max())
    Camino : []
    While Fin != -1:
        Camino.append(Fin)
        Fin = Prev[Fin]

    Return camino.reverse
```

¿P. TINDARD?

La complejidad de este problema crece linealmente con la cantidad de vertices y aristas que tenga el grafo. Por lo tanto, la complejidad es $O(V+E)$

Ignacio Vian 106930 B

) Un algoritmo greedy es un tanto mas sencillo. Buscamos solamente el optimo local, sabiendo que la solucion no va a ser la mas optima pero que tampoco va a ser la peor. En este caso, sencillamente vamos viendo a para en un vertice, ver que camino es mas barato y avanzar por ahí ✓

```
def greedy_camino(grafo)
    camino: [ ]
    vertice
    vertice = Ezeiza
    while vertice != Bariloche:
        costo_min = ∞
        For vecino in vertice.adj():
            if vecino.Peso() < costo_min:
                opt = vecino
                costo_min = vecino.Peso()
        camino.append(vertice)
        vertice = opt
    return camino
```

Como mencioné antes, la solución no es optima ya que no tenemos por todos los caminos en busqueda de la mejor globalmente sino que solo optimizamos las decisiones locales.

ejemplo?

complejidad?

Ignacio Vian 106930

1) [illegible]

backtracking o fuerza bruta con poda. La poda consiste o en encontrar una solución mejor o la actual o en restringir aquellas opciones que no cumplen las condiciones

```
def s_valida (Solucion, Fila, Col):
    For i in range (Fila)
X       if solucion[i]==col or solucion[i]-col == solucion[i]-Fila
            return false
    return True

def reinas ():
    Sol = [-1] * n
    Soluciones = []
```

(diagonale)

```
def bt (Fila)
    if Fila == n:
        Soluciones.append (solucion[:])
        return
    For col in range (8):
        if s_valida (solucion, Fila, col):
            Solucion[Fila] = col
            bt (Fila + 1)
            Solucion[Fila] = -1
```

```
bt (0)
Return Soluciones
```

Poda: chequea de columna y diagonales cuando uso s_valida()

ROSSI

(b)

(2)

Para comenzar a plantear mi Regla Greedy, asumo que mi set de Datos es un Array donde cada posición Representa un Día Distinto y en la cada posición Tengo como Valor el Beneficio (B_i), Para ello, Voy a ordenar la Lista de forma Decendente para Tener en los primeros 20 Días los mayores Beneficios, Para luego ir tomando de a Dos Días Seguidos y Descontar 1. La Solución No Sera del Todo optima, ya que Al ordenar el Array de forma Decendente Trabajar 2 Días el Día que marcos Descanse estara Perdiendo una cantidad Considerable de Beneficio, Pero Podria compensarlo o no con los Proximos Dos Días de Trabajo

97190	FRANCO BRAGANTINI	HOJA 1/4
		FECHA

B

2) GREEDY

Voy a asumir que tengo Changos, cada uno tiene un beneficio y un dia asociado (del 1 al 20)
Los voy a ordenar descendentemente por el beneficio de cada uno e ir chequeando que se cumplan las restricciones, una changa por dia y no mas de 2 dias seguidos trabajando

```
def Changos-20-dias()
    changos = Array[N]  # N es la cantidad de changos
    dias = Array[20]  # lleno de NULL al principio
    ordenar_por_beneficio(changos)
    changa in changos do  # itero arreglo de changos
        si dias[changa.dia] [redacted] == NULL
            si no_mas_de_dos_dias_seguidos(changa, dias)
                dias[changa.dia] = changa
    return dias

# no_mas_de_dos_dias_seguidos se fija si agre=
# gando esta changa que le paso por parámetro,
# no rompo la restriccion, fijandose los 2 dias an=
# teriores y 2 dias posteriores al dia en cuestion
# (son varios casos a chequear y es algo mecánico)
```

Esta solucion es óptima siempre

NOTA

97190 FRANCO BRAGANTINI HOJA 2/4 FECHA

(MB) 3) P.L.

$$y_{ni} \begin{cases} 1 & \text{si se realiza el trabajo 'i' del día 'n'} \\ 0 & \text{sino} \end{cases}$$

B_{ni} → Beneficio de realizar el trabajo i del día n

La función objetivo es maximizar el beneficio obtenido en los siguientes 20 días

$$\text{Max}(z) = \sum_n \sum_i y_{ni} \cdot B_{ni}$$, donde n varía de 1 a 20, mientras que i → la cantidad de trabajos por día, que aunque es variable, se podría tomar el valor del día con más trabajos, y para los días que se tengan menos cantidad, el beneficio valga 0 (para no escribir uno por uno los términos)

Restricciones:

$$\sum_i y_{ni} + \sum_i y_{n+1\,i} + \sum_i y_{n+2\,i} \leq 2$$

no se puede trabajar más de 2 días seguidos, esto con n de 1 a 18 (casi se llega al día 20)

$$\sum_i y_{ni} \leq 1$$

solo realiza una de las propuestas del día (o ninguna) para todo n

NOTA

97190 FRANCO BRAGANTINI HOJA Nº 3/4 FECHA

4) D & C

R

```
def menor_no_induido(arr, low, high)
    si low > high
        return low + 1

    mid = (low + high) / 2
    si arr[mid] > mid          NO FUNCIONA SI LAS SUBLISTAS NO CAMBIAN  1
        return menor_no_induido(arr, mid+1, high)
    sino
        return menor_no_induido(arr, low, mid)
```

Tengo 2 llamadas recursivas, dividiendo el array en 2 mitades. Luego no recorro el array, solo calculos matemáticos.

Por ende, la complejidad es $O(n^1)$ ó $O(n)$ X NO

NOTA

Ignacio Vian 190930

5) Prog lineal: elija el ejercicio (4) (B)

Variables

$x_{i,j} \begin{cases} = 1 & \text{si hay una reina en la fila } i, \text{ columna } j \\ = 0 & \text{si no} \end{cases}$

$i, j \in \{0, \dots, 7\}$

Función objetivo:

Max $\sum_{i=0}^{7} \sum_{j=0}^{7} x_{i,j}$

Restricciones:

Debe haber una reina por fila:

$$\sum_{j=0}^{7} x_{i,j} = 1 \quad \forall i \in \{0, \dots, 7\}$$

Debe haber una reina por columna:

$$\sum_{i=0}^{7} x_{i,j} = 1 \quad \forall j \in \{0, \dots, 7\}$$

Esto + la condición cumplida, una reina por diagonal:

mentira, mas o ver las condiciones, una por cada diagonal

$$\sum_{i,j: i+j=d} x_{i,j} <= 1 \quad \forall d \in \{-6, \dots, 6\}$$

TODAS LAS DIAGONALES NO SOLO LAS PRINCIPALES

$$\sum_{i,j: i-j=d} x_{i,j} <= 1 \quad \forall d \in \{-6, \dots, 6\}$$